МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

ІКТА

Кафедра ЗІ



ЗВІТ

До лабораторної роботи № 1

З курсу: “Програмування скриптовими мовами”

На тему: *“*Основи розробки на Python, Git та стандарти стилю коду*“*

Виконав: ст. гр. КБ-204 Тертичний М.Д.

Прийняв: Кебус В.М.

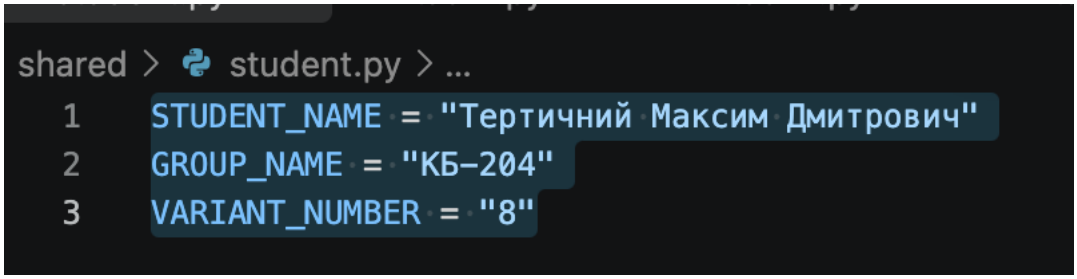
Львів – 2026

Мета

1. Ознайомитися з основами роботи з розподіленою системою контролю версій Git, платформою GitHub та створенням ізольованих віртуальних середовищ (virtualenv).
2. Навчитись створювати та структурувати проекти на Python для забезпечення багаторазового використання коду (модульність).
3. Опанувати стандарти оформлення коду PEP-8 та навчитись перевіряти код за допомогою сучасного швидкого статичного аналізатора Ruff.
4. Закріпити навички роботи з умовними операторами, циклами, функціями, колекціями (списки, кортежі, словники, множини), обробкою винятків та збереженням даних (CSV, JSON) у контексті прикладних задач кібербезпеки.

Код:

student.py



main.py

```python
import os
import sys

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../../')))

from labs.lab01 import task1, task2, task3


def main():
    print("=" * 50)
    print("Завдання 1: Аналізатор надійності паролів")
    print("=" * 50)
    task1.main()

    print("\n" + "=" * 50)
    print("Завдання 2: Система контролю доступу")
    print("=" * 50)
    task2.main()

    print("\n" + "=" * 50)
    print("Завдання 3: Хешування, CSV та JSON-логування")
    print("=" * 50)
    task3.main()


if __name__ == "__main__":
    main()
```

task1.py:

```python
import os
import random
import sys

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../../')))
from shared.student import GROUP_NAME, STUDENT_NAME, VARIANT_NUMBER

passwords = [
    "ThreatH@nt3r", "weak123", "P3n3trat10n@Test", "visitor",
    "Cyber@Defense2023", "normal", "Incident@R3sp0nse", "standard",
    "Risk@Analys1s", "typical",
]

criteria = {
    "min_length": 10,
    "require_digits": True,
    "require_upper": True,
    "require_special": True,
}

forbidden_passwords = {"weak123", "visitor", "normal", "standard", "typical", "admin"}


def add_duplicates(password_list):

    duplicates = [random.choice(password_list) for i in range(3)]
    # print(duplicates)
    return password_list + duplicates


def assess_password(password, min_length, all_passwords):

    if password in forbidden_passwords or len(password) < min_length:
        return "Заборонений"

    has_digit = any(c.isdigit() for c in password)
    has_upper = any(c.isupper() for c in password)
    has_lower = any(c.islower() for c in password)
    has_special = any(not c.isalnum() for c in password)

    groups_met = sum([has_digit, has_upper, has_lower, has_special])
    meets_all_required = has_digit and has_upper and has_special

    if meets_all_required:
        is_long_enough = len(password) >= min_length + 4
        is_unique = all_passwords.count(password) == 1
        if is_long_enough and is_unique:
            return "Дуже сильний"
        return "Сильний"

    if groups_met >= 2:
        return "Середній"

    return "Слабкий"


def analyze_passwords(password_list, min_length):

    results = []
    for password in password_list:
        strength = assess_password(password, min_length, password_list)
        results.append((password, strength))
    return results
```

```
64
65
66      def print_results(results):
67
68          for password, strength in results:
69              print(f"{password:<20} --> {strength:<15}")
70
71
72      def main():
73          print(f"{STUDENT_NAME}, {GROUP_NAME}, Варіант: {VARIANT_NUMBER}")
74          full_list = add_duplicates(passwords)
75          # print(full_list)
76          results = analyze_passwords(full_list, criteria["min_length"])
77          print_results(results)
78
79
80      if __name__ == "__main__":
81          main()
```

task2.py:

```
1   import os
2   import sys
3
4   sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../../')))
5
6   from shared.student import GROUP_NAME, STUDENT_NAME, VARIANT_NUMBER
7
8   users = {
9       "crypto_specialist": {"role": "cryptographer", "clearance": 4, "department": "Cryptography", "active": True},
10      "privacy_officer": {"role": "privacy_analyst", "clearance": 3, "department": "Privacy", "active": True},
11      "data_scientist": {"role": "data_analyst", "clearance": 2, "department": "Analytics", "active": True},
12      "field_engineer": {"role": "field_support", "clearance": 2, "department": "Field Ops", "active": True},
13      "test_account": {"role": "testing", "clearance": 1, "department": "QA", "active": False},
14  }
15
16  resources = [
17      ("encryption_keys", 4), ("privacy_policies", 3), ("anonymized_data", 2),
18      ("field_reports", 2), ("crypto_algorithms", 4), ("consent_forms", 1),
19      ("data_classification", 3), ("key_management", 4), ("statistical_models", 2),
20      ("public_datasets", 1),
21  ]
22
23  security_levels = ("Unclassified", "For Official Use", "Confidential", "Secret")
24  blocked_users = {"test_account", "gdpr_violation", "data_breach_user"}
25
26
27  def resource_level_name(level):
28      return security_levels[level - 1]
29
30
31  def print_resources():
32      print("Ресурси системи:")
33      for name, level in resources:
```

```
        print(f"  {name} -> {resource_level_name(level)}")


def check_access(username, resource_level):
    if username not in users:
        return False, "User not found"

    if username in blocked_users:
        return False, "User is blocked"

    user = users[username]

    if not user["active"]:
        return False, "Account inactive"

    if user["clearance"] < resource_level:
        return False, "Insufficient clearance"

    return True, ""


def check_all_access():
    results = []
    for username in users:
        for resource_name, resource_level in resources:
            allowed, reason = check_access(username, resource_level)
            results.append((username, resource_name, allowed, reason))
    return results


def print_access_results(results):
    for username, resource_name, allowed, reason in results:
        status = "ALLOW" if allowed else f"DENY ({reason})"
        print(f"user={username} resource={resource_name} -> {status}")


def main():
    print(f"{STUDENT_NAME}, {GROUP_NAME}, Варіант: {VARIANT_NUMBER}")
    print_resources()
    print()
    results = check_all_access()
    print_access_results(results)


if __name__ == "__main__":
    main()
```

task3.py:

```python
import csv
import datetime
import hashlib
import json
import os
import sys

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../../')))

from shared.student import GROUP_NAME, STUDENT_NAME, VARIANT_NUMBER


class ValidationError(Exception):
    """Виникає, якщо пароль коротший за мінімальну довжину."""

MIN_PASSWORD_LENGTH = 11
PERSONAL_SALT = str(VARIANT_NUMBER).zfill(5)

DATA_DIR = os.path.join(os.path.dirname(__file__), "data")
USERS_CSV = os.path.join(DATA_DIR, "users.csv")
LOG_JSON = os.path.join(DATA_DIR, "log.json")

users_to_register = (
    ("alice", "Str0ngP@ssword1"),
    ("bob", "An0therSecur3!"),
    ("carol", "MyP@ssphrase99"),
    ("dave", "S3cur3P@ss2023"),
    ("erin", "C0mplex#Pass1"),
    ("frank", "Sh0rt"),
    ("grace", "Pr0tect3d@Data"),
    ("heidi", "Vault!K3eper9"),
    ("ivan", "Cyb3r@Def3nse"),
    ("judy", "L0ckdown#2023"),
)


def generate_hash(password: str, salt: str = "00000") -> str:
    if not password or not salt:
        raise ValueError("Пароль та сіль не можуть бути порожніми")
    if len(password) < MIN_PASSWORD_LENGTH:
        raise ValidationError(
            f"Пароль коротший за мінімальну довжину ({MIN_PASSWORD_LENGTH})"
        )
    return hashlib.sha256((password + salt).encode()).hexdigest()


def create_user(username, password):
    hash_value = generate_hash(password, PERSONAL_SALT)
    return username, hash_value


def create_users(users_list):
    os.makedirs(DATA_DIR, exist_ok=True)
    with open(USERS_CSV, "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        for username, password in users_list:
            try:
                username, hash_value = create_user(username, password)
                writer.writerow([username, hash_value])
            except ValidationError as error:
                print(f"Пропущено {username}: {error}")


def read_users_db():
    users_db = []
```

```
    with open(USERS_CSV, "r", encoding="utf-8") as file:
        reader = csv.reader(file)
        for row in reader:
            users_db.append(tuple(row))
    return users_db


def print_users_db(users_db):
    print(f"{'Логін':<15} {'Хеш':<70}")
    print("-" * 85)
    for username, hash_value in users_db:
        print(f"{username:<15} {hash_value:<70}")


def log_event(func):

    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)

        entry = {
            "event": "login",
            "user": args[0] if args else kwargs.get("username", ""),
            "result": "success" if result else "failure",
            "timestamp": datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "args": list(args),
            "kwargs": kwargs,
        }

        os.makedirs(DATA_DIR, exist_ok=True)
        logs = []
        if os.path.exists(LOG_JSON):
            with open(LOG_JSON, "r", encoding="utf-8") as file:
                try:
                    logs = json.load(file)
                except json.JSONDecodeError:
                    logs = []

            logs.append(entry)
            with open(LOG_JSON, "w", encoding="utf-8") as file:
                json.dump(logs, file, indent=2, ensure_ascii=False)

            return result

    return wrapper


@log_event
def login(username: str, password: str) -> bool:

    if not username or not password:
        raise ValueError("Логін та пароль не можуть бути порожніми")

    hash_value = generate_hash(password, PERSONAL_SALT)
    users_db = read_users_db()

    for db_username, db_hash in users_db:
        if db_username == username and db_hash == hash_value:
            return True
    return False


def main():
```

```python
    print(f"{STUDENT_NAME}, {GROUP_NAME}, Варіант: {VARIANT_NUMBER}")

    try:
        create_users(users_to_register)
        users_db = read_users_db()
        print_users_db(users_db)

        test_username, test_password = users_to_register[0]
        result = login(test_username, test_password)
        print(f"\nВхід '{test_username}' з правильним паролем: {'успішно' if result else 'невдало'}")

        result_wrong = login(test_username, "wrong_password_here")
        print(f"Вхід '{test_username}' з неправильним паролем: {'успішно' if result_wrong else 'невдало'}")

    except FileNotFoundError as error:
        print(f"Файл не знайдено: {error}")
    except PermissionError as error:
        print(f"Немає доступу до файлу: {error}")
    except OSError as error:
        print(f"Помилка вводу-виводу: {error}")
    except ValidationError as error:
        print(f"Помилка валідації: {error}")
    except ValueError as error:
        print(f"Помилка значення: {error}")


if __name__ == "__main__":
    main()
```

Лістинг всіх завданнь:

==========

Завдання 1: Аналізатор надійності паролів

==========

Тертичний Максим Дмитрович, КБ-204, Варіант: 8

ThreatH@nt3r --> Сильний

weak123 --> Заборонений

P3n3trat10n@Test --> Дуже сильний

visitor --> Заборонений

Cyber@Defense2023 --> Дуже сильний

normal --> Заборонений

Incident@R3sp0nse --> Дуже сильний

standard --> Заборонений

Risk@Analys1s --> Сильний

typical --> Заборонений

weak123 --> Заборонений

weak123 --> Заборонений

weak123 --> Заборонений

==========

Завдання 2: Система контролю доступу

==========

Тертичний Максим Дмитрович, КБ-204, Варіант: 8

Ресурси системи:

encryption_keys -> Secret

privacy_policies -> Confidential

anonymized_data -> For Official Use

field_reports -> For Official Use

crypto_algorithms -> Secret

consent_forms -> Unclassified

data_classification -> Confidential

key_management -> Secret

statistical_models -> For Official Use

public_datasets -> Unclassified

user=crypto_specialist resource=encryption_keys -> ALLOW

user=crypto_specialist resource=privacy_policies -> ALLOW

user=crypto_specialist resource=anonymized_data -> ALLOW

user=crypto_specialist resource=field_reports -> ALLOW

user=crypto_specialist resource=crypto_algorithms -> ALLOW

user=crypto_specialist resource=consent_forms -> ALLOW

user=crypto_specialist resource=data_classification -> ALLOW

user=crypto_specialist resource=key_management -> ALLOW

user=crypto_specialist resource=statistical_models -> ALLOW

user=crypto_specialist resource=public_datasets -> ALLOW

user=privacy_officer resource=encryption_keys -> DENY (Insufficient clearance)

user=privacy_officer resource=privacy_policies -> ALLOW

user=privacy_officer resource=anonymized_data -> ALLOW

user=privacy_officer resource=field_reports -> ALLOW

user=privacy_officer resource=crypto_algorithms -> DENY (Insufficient clearance)

user=privacy_officer resource=consent_forms -> ALLOW

user=privacy_officer resource=data_classification -> ALLOW

user=privacy_officer resource=key_management -> DENY (Insufficient clearance)

user=privacy_officer resource=statistical_models -> ALLOW

user=privacy_officer resource=public_datasets -> ALLOW

user=data_scientist resource=encryption_keys -> DENY (Insufficient clearance)

user=data_scientist resource=privacy_policies -> DENY (Insufficient clearance)

user=data_scientist resource=anonymized_data -> ALLOW

user=data_scientist resource=field_reports -> ALLOW

user=data_scientist resource=crypto_algorithms -> DENY (Insufficient clearance)

user=data_scientist resource=consent_forms -> ALLOW

user=data_scientist resource=data_classification -> DENY (Insufficient clearance)

user=data_scientist resource=key_management -> DENY (Insufficient clearance)

user=data_scientist resource=statistical_models -> ALLOW

user=data_scientist resource=public_datasets -> ALLOW

user=field_engineer resource=encryption_keys -> DENY (Insufficient clearance)

user=field_engineer resource=privacy_policies -> DENY (Insufficient clearance)

user=field_engineer resource=anonymized_data -> ALLOW

user=field_engineer resource=field_reports -> ALLOW

user=field_engineer resource=crypto_algorithms -> DENY (Insufficient clearance)

user=field_engineer resource=consent_forms -> ALLOW

user=field_engineer resource=data_classification -> DENY (Insufficient clearance)

user=field_engineer resource=key_management -> DENY (Insufficient clearance)

user=field_engineer resource=statistical_models -> ALLOW

user=field_engineer resource=public_datasets -> ALLOW

user=test_account resource=encryption_keys -> DENY (User is blocked)

user=test_account resource=privacy_policies -> DENY (User is blocked)

user=test_account resource=anonymized_data -> DENY (User is blocked)

user=test_account resource=field_reports -> DENY (User is blocked)

user=test_account resource=crypto_algorithms -> DENY (User is blocked)

user=test_account resource=consent_forms -> DENY (User is blocked)

user=test_account resource=data_classification -> DENY (User is blocked)

user=test_account resource=key_management -> DENY (User is blocked)

user=test_account resource=statistical_models -> DENY (User is blocked)

user=test_account resource=public_datasets -> DENY (User is blocked)

==========

Завдання 3: Хешування, CSV та JSON-логування

==========

Тертичний Максим Дмитрович, КБ-204, Варіант: 8

Пропущено frank: Пароль коротший за мінімальну довжину (11)

Логін Хеш

---

alice
166524edf2e59234f7fbcaeb1e0410375504fb28bc6e741db1fd7fe876be90bf

bob
e379ce796c5d353c57b5fb26183d4b4322ae11c7ae4c62bb3123719b8ff7b590

carol
74f74bb77aab02200c9b347d3fbe092146be1568291d027adfe1ec592c438900

dave
3facf58dbc43d2315f521e8a0961e9a69d30fa58a104d8b0378e9c1130c66a86

erin
d05cd8b6716ba927bf348547da491f934c875612bc6044da6f0387f44d6472a2

grace
b1c7b95f3a05c075fade709b9b5ba616683a71bacadd31dc1ff9389110bb730c

heidi
25ad024f4946e62507558ec4153ba98c392a0feeb828bbc5f5e1d34aeeed0e92

ivan
8a83be9290456301f2c5c02ad051c33fb27c6da6173fa44e0e335042342f951e

judy
d50ea53218ff4cbdd8c4ce06dd5cfc71c8ab61218b8befd1636b6ce1ea9ca530

Вхід 'alice' з правильним паролем: успішно

Вхід 'alice' з неправильним паролем: невдало

Файл users.csv:

labs > lab01 > data > users.csv

```
alice,166524edf2e59234f7fbcaeb1e0410375504fb28bc6e741db1fd7fe876be90bf
bob,e379ce796c5d353c57b5fb26183d4b4322ae11c7ae4c62bb3123719b8ff7b590
carol,74f74bb77aab02200c9b347d3fbe092146be1568291d027adfe1ec592c438900
dave,3facf58dbc43d2315f521e8a0961e9a69d30fa58a104d8b0378e9c1130c66a86
erin,d05cd8b6716ba927bf348547da491f934c875612bc6044da6f0387f44d6472a2
grace,b1c7b95f3a05c075fade709b9b5ba616683a71bacadd31dc1ff9389110bb730c
heidi,25ad024f4946e62507558ec4153ba98c392a0feeb828bbc5f5e1d34aeeed0e92
ivan,8a83be9290456301f2c5c02ad051c33fb27c6da6173fa44e0e335042342f951e
judy,d50ea53218ff4cbdd8c4ce06dd5cfc71c8ab61218b8befd1636b6ce1ea9ca530

```

Файл log.json:


```
[
  {
    "event": "login",
    "user": "alice",
    "result": "success",
    "timestamp": "2026-09-21 10:37:01",
    "args": [
      "alice",
      "Str0ngP@ssword1"
    ],
    "kwargs": {}
  },
  {
    "event": "login",
    "user": "alice",
    "result": "failure",
    "timestamp": "2026-09-21 10:37:01",
    "args": [
      "alice",
      "wrong_password_here"
    ],
    "kwargs": {}
  },
  {
    "event": "login",
    "user": "alice",
    "result": "success",
    "timestamp": "2026-09-21 10:38:19",
    "args": [
      "alice",
      "Str0ngP@ssword1"
    ],
```


Та далі логи

Ruff check:



Висновки:

У ході виконання лабораторної роботи №1 було освоєно базові інструменти розробки на Python у контексті кібербезпеки. Налаштовано ізольоване віртуальне середовище (venv) та структуровано репозиторій за принципом DRY — персональні дані винесено в окремий модуль shared/student.py, що уникає дублювання коду в наступних роботах. Код перевірено та відформатовано за допомогою лінтера Ruff відповідно до стандарту PEP-8.

У межах індивідуальних завдань варіанту 8 реалізовано: аналізатор надійності паролів з оцінкою за п'ятьма рівнями складності; багаторівневу систему контролю доступу з перевіркою за клірансом, блокуванням та активністю облікового запису; модуль безпечної автентифікації з хешуванням паролів (SHA-256 із персональною сіллю), збереженням бази користувачів у CSV та журналюванням спроб входу у JSON через декоратор @log_event, з обробкою специфічних винятків (ValidationError, ValueError, FileNotFoundError тощо).

Практично закріплено роботу зі списками, кортежами, множинами та словниками, а також розуміння того, чому вибір конкретної колекції (наприклад, set для заблокованих користувачів) впливає на продуктивність системи при масштабуванні.